

REALMATCH+

채팅 설계서

1:1 채팅방, 메시지 구조, 읽음 처리, 차단/신고 연동, 실시간 전달 구조를 중심으로 정리한 초기 설계 문서

문서 목적	로그인·회원·사용자정보 및 매칭·추천 설계 이후, 매칭 성사 사용자 간 커뮤니케이션 구조를 별도 설계 기준으로 사용
적용 범위	1:1 채팅방 생성, 메시지 송수신, 읽음 처리, 차단·신고 연동, 채팅방 상태값, 실시간 전달 구조
설계 원칙	매칭 도메인과 채팅 도메인을 분리하고, 실시간성과 데이터 정합성을 동시에 확보
초기 산출물	채팅 정책 초안, 핵심 테이블 구조, 상태값 정의, 실시간 처리 포인트, 핵심 API 범위

1. 채팅 영역 설계 목표

- 모바일 앱 기준으로 매칭 성사 이후 사용자가 자연스럽게 대화로 이어질 수 있도록 1:1 채팅 흐름을 일관된 구조로 관리한다.
- 초기 MVP 단계에서는 텍스트 중심 채팅과 읽음 처리, 차단·신고 연동을 우선 적용하되, 이후 이미지·음성·시스템 메시지 확장이 가능하도록 설계한다.
- 채팅방, 메시지, 읽음 상태를 분리해 매칭 도메인과 커뮤니케이션 도메인이 뒤섞이지 않도록 한다.

2. 채팅방 생성 방식 및 정책

- 기본 채팅 단위는 1:1 채팅방이며, 상호 호감으로 매칭이 성사된 경우 자동 생성한다.
- 하나의 match_id 에 대해 하나의 chat_room 만 유지하며, 동일 매칭에서 중복 채팅방을 생성하지 않는다.
- 차단·신고·매칭 종료 상태가 발생하면 채팅방 상태를 함께 갱신해 메시지 전송 가능 여부를 일관되게 관리한다.
- 향후 랜덤통화 기능이 연결되더라도 통화 세션과 채팅방은 별도 도메인으로 유지해 일반 채팅과 실시간 연결을 분리한다.

2-1. 채팅 사용 제한 원칙

항목	정책	이유
차단 관계 사용자	상호 메시지 전송 즉시 제한	안전성 확보
신고 검토 중 사용자	운영 정책에 따라 전송 제한 또는 모니터링	악성 사용자 대응
종료된 매칭	기본적으로 신규 메시지 전송 제한	도메인 일관성 유지
비활성 채팅방	목록 조회만 허용하고 전송 차단	상태 관리 단순화

3. 메시지 구조 및 정책

- 메시지 타입은 TEXT, IMAGE, SYSTEM 을 기본값으로 두고 시작한다.
- SYSTEM 메시지는 매칭 성사, 채팅방 생성, 차단, 종료 안내 등 서비스 상태 전달용으로 사용한다.
- 메시지는 반드시 DB 저장을 기준으로 하며, 실시간 전달은 저장 이후에 수행한다.
- 초기 정책은 메시지 수정·삭제보다 송신/수신/읽음 흐름을 우선 확정하고, 운영 로그와 신고 연계를 고려한다.

4. 핵심 데이터 모델

4-1. 채팅 도메인 테이블

테이블	주요 컬럼	설명
chat_rooms	room_id, match_id, room_type, status, created_at	매칭 기준으로 생성되는 1:1 채팅방 정보를 관리
chat_messages	message_id, room_id, sender_id, message_type, content, created_at	채팅 메시지 본문과 타입, 송신자 정보를 저장
chat_read_status	room_id, user_id, last_read_message_id, read_at	읽음 처리 기준점과 unread count 계산 정보를 관리
chat_room_members	room_id, user_id, joined_at, left_at	채팅방 참여 사용자와 이탈 시점을 보조 관리
chat_message_reports	message_id, reporter_id, reason, status	메시지 단위 신고와 운영 검토 상태를 저장

4-2. 상태값 정의

구분	상태값	설명
채팅방 상태	ACTIVE	현재 정상 대화 가능 상태
채팅방 상태	CLOSED	종료된 채팅방
채팅방 상태	BLOCKED	차단 또는 운영 제한으로 비활성화된 상태
메시지 상태	SENT	서버 저장 완료
메시지 상태	DELIVERED	수신 사용자 세션에 전달됨
메시지 상태	READ	수신 사용자가 읽음 처리 완료

5. 채팅 흐름

- 매칭 성사: 상호 호감 매칭이 생성되면 chat_room 을 자동 생성한다.
- 메시지 전송: 사용자가 메시지를 전송하면 서버는 유효한 room 상태와 차단 여부를 확인한 뒤 DB 에 저장한다.
- 실시간 전달: 저장 완료된 메시지를 WebSocket 채널을 통해 상대 사용자에게 전달한다.
- 읽음 처리: 채팅방 진입 또는 메시지 조회 시 마지막 읽은 메시지 기준을 갱신한다.
- 상태 변경: 차단·신고·매칭 종료가 발생하면 채팅방 상태와 메시지 전송 가능 여부를 함께 갱신한다.

6. 핵심 API 범위

구분	API	역할
채팅방 목록	GET /api/v1/chat/rooms	내 채팅방 목록과 unread count 조회
채팅방 상세	GET /api/v1/chat/rooms/{roomId}	채팅방 메시지 이력 조회
메시지 전송	POST /api/v1/chat/rooms/{roomId}/messages	메시지 저장 및 실시간 전달 처리
읽음 처리	POST /api/v1/chat/rooms/{roomId}/read	마지막 읽은 메시지 기준 갱신
채팅방 종료	POST /api/v1/chat/rooms/{roomId}/close	사용자 종료 또는 상태 전환 처리

7. Redis 확장 포인트

- 초기 채팅 저장은 DB 중심으로 운영하되, 접속 세션과 메시지 라우팅은 Redis 를 보조 계층으로 사용할 수 있다.
- 예: chat:session:{userId} 로 현재 접속 세션을 관리하고, chat:room:unread:{roomId}:{userId} 형태로 unread 집계를 보조할 수 있다.
- 실시간 전송 보조를 위해 publish/subscribe 채널 또는 세션 매핑 캐시를 활용할 수 있다.

8. 보안 및 운영 기준

- 메시지 전송 API 는 차단 관계와 채팅방 상태를 항상 우선 검증해 안전성을 보장한다.
- 읽음 처리와 메시지 전송은 중복 요청에도 일관된 결과를 유지하도록 idempotent 하게 설계한다.
- 메시지 본문은 금치어·신고 연동·운영 로그 추적이 가능하도록 저장 정책을 정의한다.
- 채팅 데이터는 개인정보 최소 원칙과 보관 기간 기준에 맞춰 관리하고, 운영 목적 외 사용을 제한한다.

9. 정리

- 본 문서는 REALMATCH+ 전체 제안서 중 채팅 영역만 독립적으로 정리한 설계 기준 문서다.
- 핵심은 매칭 이후 사용자가 어떤 채팅방에서 어떤 상태로 대화하고, 메시지가 어떻게 저장·전달·읽힘 처리되는지를 구조적으로 분리하는 것이다.
- 이 구조를 먼저 확정하면 이후 랜덤통화, 알림, 신고·운영 기능 연계를 안정적으로 확장할 수 있다.